

Intensive Programming Course

Exercise 1.1

1. Test IPython in interactive mode

1.1. Opening IPython in interactive mode

Open a command line window (Command-line, Terminal, or Shell).

In the command line window, enter

```
ipython
```

and then press Enter.

You should now see text like this. The actual output will vary depending on your platform and the version of IPython you have installed.

```
Python 3.14.7 (v3.14.7:823f0323ee6, Aug  5 2026, 07:07:01) [Clang 21.0.0 (clang-2100.1.1.101)]
```

```
Type 'copyright', 'credits' or 'license' for more information
```

```
IPython 9.17.1 -- An enhanced Interactive Python. Type '?' for help.
```

```
Tip: You can change the editing mode of IPython to behave more like vi, or emacs.
```

```
In [1]:
```

The text "In [1]:" is a prompt indicating that IPython is waiting for your input. You can type ? for help or start typing snippets, as you will do in a moment.

1.2. Evaluating expressions (snippets)

Enter the following expression at the prompt and finish typing with the Enter key.

```
In [1]: 45 + 72
```

```
Out[1]: 117
```

```
In [2]:
```

After you enter `45 + 72` and press Enter, IPython reads the snippet, evaluates it, and outputs its result (117) in `Out[1]`. IPython then displays the prompt `In [2]` to indicate that it is waiting for you to enter your second snippet. For each new snippet, IPython increases the number in square brackets by 1.

1.3. Exiting interactive mode

Enter the command

```
exit
```

in the current prompt `In [2]` and press Enter to exit the mode immediately.

Alternatively, you can also press the key combination `Ctrl + d` (or `control + d`). This will display the prompt "Are you sure you want to exit ([y]/n)?" The square brackets around [y] indicate that this is the default answer. If you now press the Enter key, the default answer will be sent and the program will exit.

MacOS and Linux only: Press `Ctrl + d` (or `control + d`) twice to exit the program immediately.

2. Running a Python program with the IPython interpreter

In this section, you will run a script called *roll_die_dynamic.py*. You do not need to understand the contents of the file at this point. You may learn about this program later in the lecture.

The extension *.py* indicates that the file contains Python source code. The script *roll_die_dynamic.py* simulates the rolling of a six-sided die. It presents a colorful, animated visualization that dynamically displays the number of sides rolled on the die.

2.1. Go to the exercises01 folder

In the command line, change to the *exercises01* folder on your system, if you are not already there.

Assuming you have stored the exercise folder in the "*Documents/prg_tut*" folder, you can change to this *exercises01* folder with the following command:

- On macOS/Linux, type `cd ~/Documents/prg_tut/exercises01` and then press Enter.
- On Windows, type `cd C:\Users\YourAccount\Documents\prg_tut\exercises01` and then press Enter. Replace "YourAccount" in *the* above path with your actual account name.

2.2. Running the script

To run the script, enter the following command in the command line and then press Enter:

```
ipython roll_die_dynamic.py 6000 1
```

After a few seconds, the script displays a window with the dynamic visualization. The numbers 6000 and 1 tell this script how many times to roll the dice and how many dice to roll each time. In this case, we roll 1 die 6000 times.

With a six-sided die, the numbers 1 to 6 should each occur with "equal probability"—the probability is 1/6 or about 16.667% for each. Therefore, if we roll a die 6000 times, we expect to see each side appear about 1000 times. Like coin tossing, die rolling is random, so there could be some sides with less than 1000, some with 1000, and some with more than 1000.

Experiment with the script by changing the value 1 to 100, 1000, and 10000, i.e., rolling 100, 1000, or 10000 dice at the same time. Also try varying the number of rolls (6000 in the example).

You will notice that as the number of dice rolls increases, the frequency approaches 16.667%. This is a phenomenon known as the "law of large numbers."

3. Writing and executing code in a Jupyter Notebook

A *Jupyter* Notebook runs in an interactive, browser-based environment. You can write and execute code in a Jupyter Notebook and enhance the code with text, images, and video.

Jupyter Notebooks are widely used in the data science community and in the broader scientific community in general. They are the preferred means of conducting Python-based data analysis studies and communicating their results in a reproducible manner.

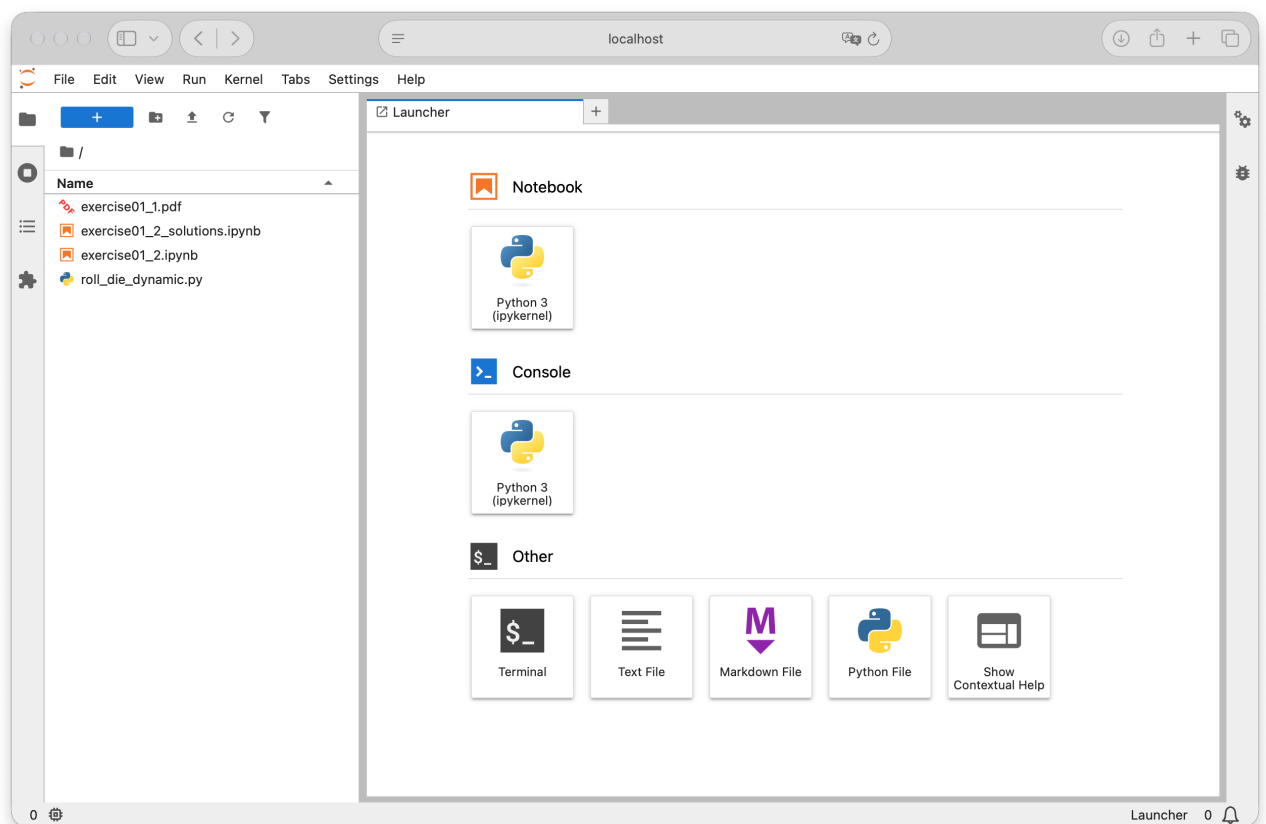
In this section, you will use the JupyterLab interface to manage your notebook files and other files that your notebooks use (such as images and videos). As you will see, *JupyterLab* also makes it convenient to write code, execute it, see the results, modify the code, and execute it again.

3.1. Opening JupyterLab in your browser

To open JupyterLab, navigate to the *exercises01* folder in your command line window (Command-line, Terminal, or Shell). Then enter the following command and press the Enter key:

```
jupyter lab
```

This will run the Jupyter Notebook server on your computer and open JupyterLab in your default web browser, displaying the contents of the *exercises01* folder in the File Browser on the left side of JupyterLab.



You can double-click files in the File Browser to open them. They will then be displayed as a separate tab.

If you accidentally close your browser, you can reopen JupyterLab by entering the following address in your web browser

`http://localhost:8888/lab`

3.2. Create a new Jupyter Notebook

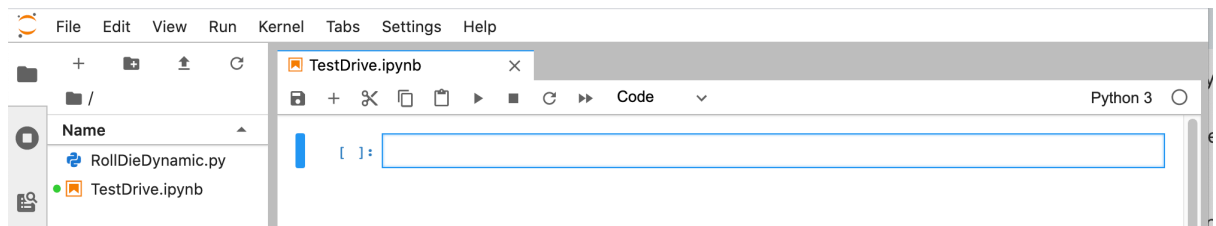
In the *Launcher* tab, click the Python 3 button to create a new Jupyter Notebook named *Untitled.ipynb*. The file extension *.ipynb* is short for IPython Notebook. You can then enter and execute Python 3 code in this Jupyter Notebook.

3.3. Rename Jupyter Notebook

Rename *Untitled.ipynb* to *TestDrive.ipynb*:

Right-click on the *Untitled.ipynb* tab and select Rename Notebook. Change the name to *TestDrive.ipynb*.

The top part of JupyterLab should now look like this:



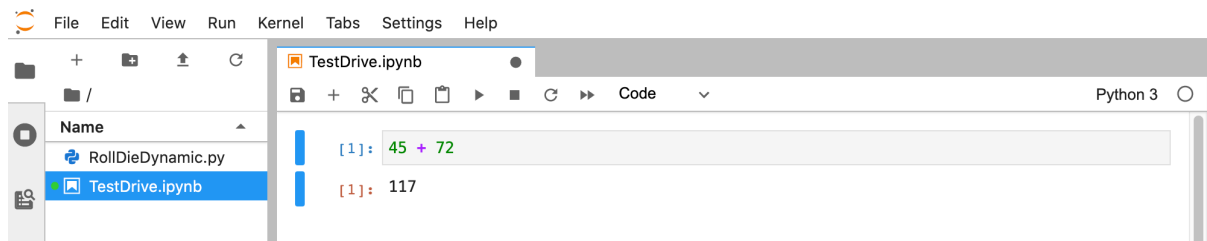
3.4. Evaluating an expression

The unit of work in a Jupyter Notebook is a cell in which you can enter code snippets. By default, a new notebook contains one cell — the rectangle in *the TestDrive.ipynb* notebook — but you can add more. To the left of the cell is the notation `[ ]:`, where the Jupyter notebook displays the number of the code snippet in the cell after you have executed the cell.

Click in the cell and then enter the following expression:

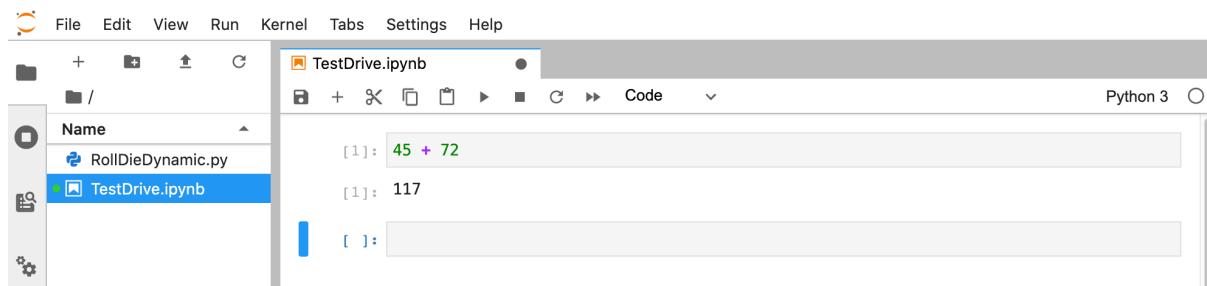
`45 + 72`

To execute the code in the current cell, type **Ctrl+Enter** (control + enter). JupyterLab executes the code in IPython and then displays the results below the cell:



3.5. Adding and executing another cell

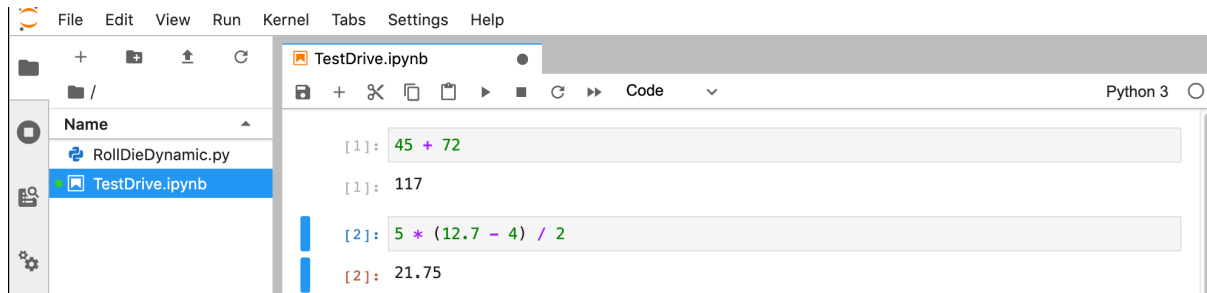
Let's evaluate another expression. First, click the + button in the toolbar above the first cell of the notebook. This adds a new cell below the current cell:



Click in the new cell and then enter the expression

`5 * (12.7 - 4) / 2`

and execute the cell by pressing **Shift+Enter** this time:



Again, the snippet is executed and the result is displayed. In addition, a new, empty cell is inserted.

3.6. Saving a Jupyter Notebook

If your notebook has unsaved changes, the X to the right of the notebook tab name changes to ●. To save the notebook, either select the File menu in JupyterLab (not at the top of the browser window) and then Save Notebook, or click the disk icon in the toolbar.

3.7. Explore JupiterLab

Familiarize yourself with JupiterLab. In the *Help* menu, you will find access to the JupiterLab documentation (via the *JupyterLab Reference* menu item), as well as other documentation and information.

3.8. Closing JupyterLab

When you are finished with JupyterLab, you can close the browser or browser tab and then enter Ctrl + c twice in the command line window (Command-line, Terminal, or Shell) from which you started JupyterLab.